BBVA AI Innovation

Editing Neural Networks with Language Models

Independent edits, cooperative search and what the measurements reveal

Davut Emre Taşar

4 November 2026 · Research report



Conceptual glass artwork · not a measurement of activations

01 · Abstract and reader map

Can language become a useful weight edit?

Neural networks normally acquire their parameters through numerical optimization. Once trained, their weights determine how an input is transformed into a prediction. Weight-space programming asks whether a language model can inspect these stored parameters and write useful changes directly. We follow that question through two controlled targets.

Abstract

UCI Adult is a census-income classification dataset: our small neural network predicts whether income exceeds $50,000 from encoded census attributes. SmolLM2 is an intact 1.7-billion-parameter base language model, evaluated through the likelihood of multiple-choice answer continuations. Sol and Astra are two configured language-model editors. They search independently from a common origin; a cooperative proposal–review stage can use their development information to propose its own intervention.

On Adult, Sol's selected patch raises balanced accuracy, the equal mean of the two class recalls, by 0.4349 percentage points. It rescues 65 positive decisions and regresses 102 negative decisions, changing ordinary accuracy by -0.2273 percentage points (a decrease). Cooperation selects a related patch but does not establish an advantage over Sol. On the intact language model, genuine nonzero proposals are evaluated, yet the selection rule retains the original weights in every condition.

Exact interventions, explicit selection and paired decisions make the independent/cooperative comparison traceable and explain what a small score change means.

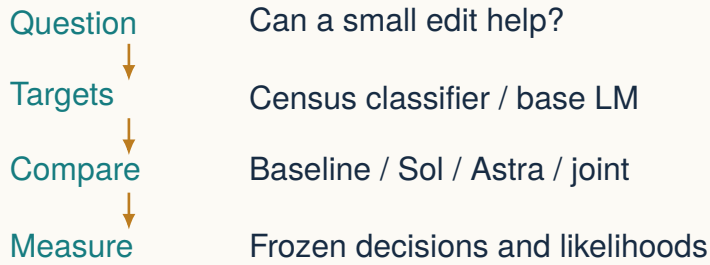
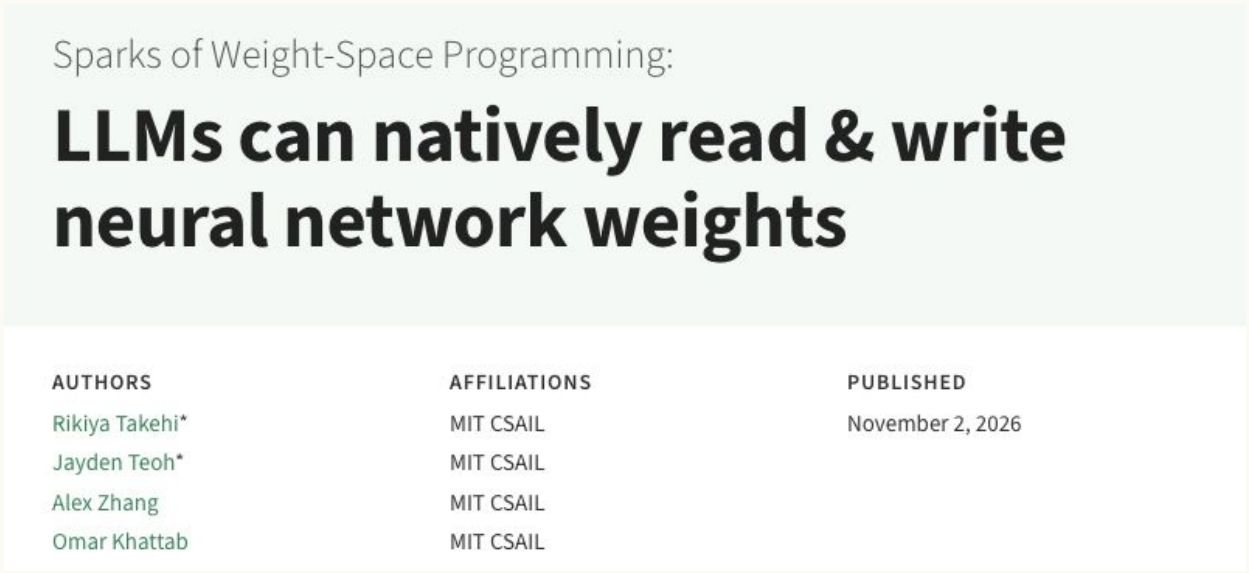


Figure 1. Reading map for our experiments. Both targets compare the unchanged baseline, independent Sol, independent Astra and cooperative joint conditions. Development selects the endpoint; frozen final measurements determine the reported outcomes.

02 · Motivation and reference

A research article opens the question



Sparks of Weight-Space Programming:

LLMs can natively read & write neural network weights

AUTHORS	AFFILIATIONS	PUBLISHED
Rikiya Takehi*	MIT CSAIL	November 2, 2026
Jayden Teoh*	MIT CSAIL	
Alex Zhang	MIT CSAIL	
Omar Khattab	MIT CSAIL	

Figure 2. The actual opening of the research blog article by Rikiya Takehi, Jayden Teoh, Alex Zhang and Omar Khattab, published 2 November 2026. This is a source-title excerpt, not a journal or arXiv cover; it motivates the experiment rather than supplying our measurements.

The reference is *Sparks of Weight-Space Programming: LLMs can natively read & write neural network weights*. Weight-space programming treats stored parameters as objects that an editor can inspect, describe and change. Instead of asking a language model only for a prediction, the system asks it to write part of the numerical rule used by another network.

Takehi and colleagues demonstrate parameter reading and writing, task recognition, policy repair and small game networks, alongside a Qwen language-model example. These demonstrations make direct editing an empirical question. A meaningful test must still specify which parameters can change, what feedback an editor receives and how a proposed improvement is selected.

Our targets and restrictions differ from the reference. We examine an independent-versus-cooperative comparison on a damaged income classifier and an intact compact language model. We do not reproduce all of the article's headline results or import its benchmark scores into our result tables.

Source

https://rikiyat.github.io/blog/weight-space-programming/

03 · Concepts before measurements

Weights persist; activations respond

A *weight* is a stored numerical coefficient. The same coefficient participates in the computation for many different inputs. A *bias* is a stored additive offset. An *activation* is an intermediate value computed for a particular input. A *tensor* is a numerical array; a matrix of weights is a two-dimensional tensor. These distinctions matter because a parameter edit changes a shared computation, while its effect on activations depends on the input.

During conventional training, an optimizer uses derivatives of a loss to update parameters. Our Adult network was trained that way. During the subsequent editing stage, the language-model editor proposes a schema-constrained intervention without gradient descent. The target performs forward computations, and the controller returns permitted development scores.

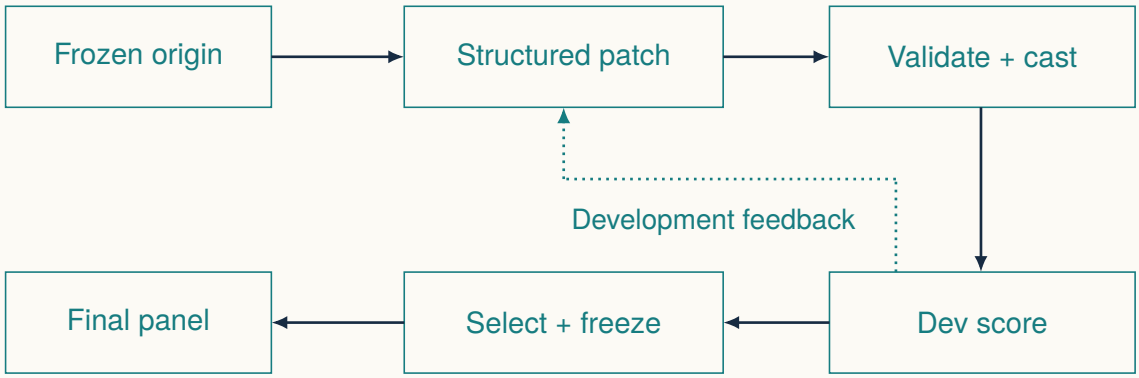


Figure 3. The direct-editing loop. Every proposal is applied to the same immutable origin, validated after casting, and evaluated on development data. The loop can return development feedback; the final panel evaluates only the frozen selection. Arrows represent information flow, not differentiation.

A *candidate* is one complete proposed intervention relative to the origin. A *patch* encodes its operations. A *selected endpoint* is the candidate retained by the development rule. Candidate zero is the unchanged origin, represented by an empty patch. An editor can propose real changes and still select zero if no changed candidate wins.

Small changes can have several outcomes: confidence can move without a decision changing; a threshold can be crossed; or a downstream nonlinear gate can suppress the difference. A legal edit is therefore a computable proposal, not yet evidence of improvement.

04 · Research questions and design

Four conditions, one starting point

We ask three questions. Can a bounded direct edit improve the selected primary metric on frozen final data? What other decisions or probability scores change with it? Does development-informed cooperation outperform the independent editor endpoints?

Baseline means no editing. Sol denotes configured editor `gpt-6.1-sol`; Astra denotes `gpt-6-astra`. Both use medium reasoning. Joint denotes the cooperative proposal–review condition using these two actors, rather than a third editor model. These are configured identities, not an inference from a model's self-description.

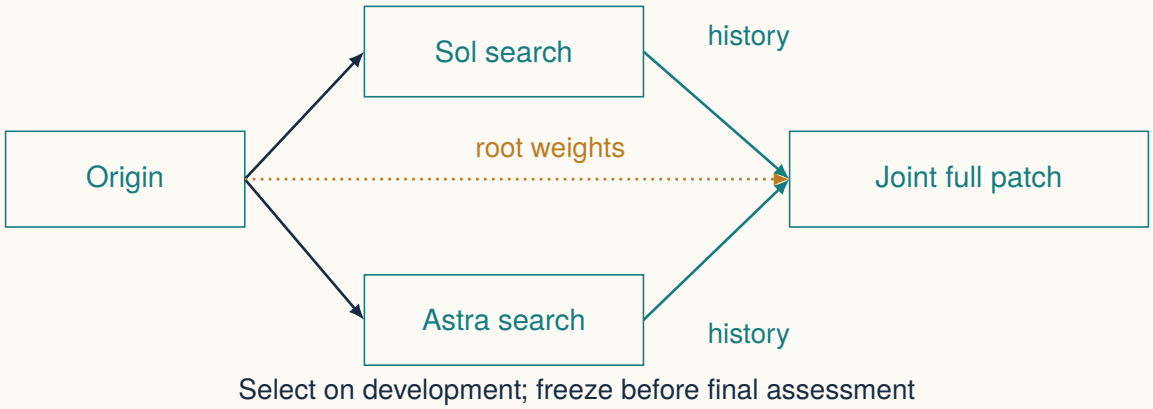


Figure 4. Experimental information flow. Sol and Astra begin from identical origin weights and retain independent histories. Joint can use both development histories but writes a full patch against that same origin. It does not apply Sol and then Astra in sequence; final answers supply no selection feedback.

A fixed rule chooses the endpoint

For Adult, selection maximizes development balanced accuracy; ties prefer a smaller executed delta norm, fewer changed scalars and then the earlier candidate index. The language-model task substitutes character-normalized development accuracy as its primary metric. Candidate zero is always eligible.

The study is one exploratory seed101 comparison. Original Adult solo allowances were ten candidates each, versus five shared joint rounds; LM v2 allowed five each. These unequal Adult allowances limit causal attribution to cooperation alone. Frozen assessment separates within-session selection from final comparison, while repeated-seed and repeated-session questions remain open.

05 · Dataset and partitioning

What the Adult classifier predicts

The UCI Adult, or Census Income, dataset derives from a 1994 census extraction. Its 48,842 original records contain 14 original input fields and a binary income target. Positive means income above $50,000; negative means income at or below it. The name Adult identifies the dataset, not a neural-network architecture.

Original fields are age, workclass, final census weight (`fnlwgt`), education, education number, marital status, occupation, relationship, race, sex, capital gain, capital loss, hours per week and native country. This experiment excludes sex and race from encoded predictors. Their exclusion does not establish demographic independence of the remaining attributes.

Partition first; fit the encoder second

The original 32,561 training records are stratified into fit (22,792), calibration (3,256) and development (6,513), using seed101. The original test split remains 16,281 rows: 3,846 positive and 12,435 negative. Fit data learn the encoder and network; calibration chooses the lesion; development guides editing; original test data assess frozen endpoints.

Within the full development partition, a fixed **2,048-row feedback panel** is actually scored per candidate. A separate fixed **256-row diagnostic panel** supports editing inspection. The 6,513 development total must not be mistaken for the number evaluated at each editing step.

Six numerical fields are standardized using fit-only means and standard deviations. Remaining categorical predictors become one-hot indicators, including a missing category; unseen categories map to all-zero indicators. The combined input has 104 dimensions. An input is a numerical representation of attributes, not a string prompt. No personal feature rows are included in this report.

06 · Network and mathematical definitions

A complete small forward computation

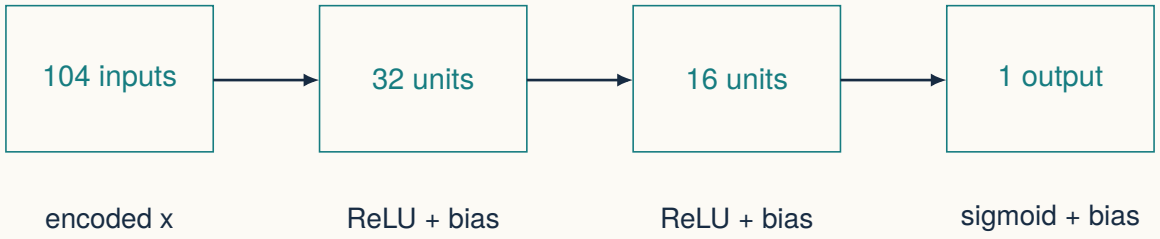


Figure 5. The Adult network maps 104 encoded inputs through hidden layers of 32 and 16 units to one positive-income probability. The diagram shows exact widths and nonlinearities, with schematic connections. All three linear layers have biases; only eligible second-layer weights are editable.

For an encoded vector $x \in \mathbb{R}^{104}$, define

$$z_i^{(1)} = \sum_{j=0}^{103} W_{ij}^{(1)} x_j + b_i^{(1)}, \qquad h_i^{(1)} = \max(0, z_i^{(1)}),$$

$$z_r^{(2)} = \sum_{i=0}^{31} W_{ri}^{(2)} h_i^{(1)} + b_r^{(2)}, \qquad h_r^{(2)} = \max(0, z_r^{(2)}),$$

$$\ell = \sum_{r=0}^{15} W_{0r}^{(3)} h_r^{(2)} + b_0^{(3)}, \qquad p = \frac{1}{1 + e^{-\ell}}.$$

Here j indexes inputs, i indexes the 32 first hidden units and r indexes the 16 second hidden units. $W^{(1)}, W^{(2)}, W^{(3)}$ are weight matrices with shapes 32×104, 16×32, 1×16. Each b is the matching bias vector. Each z is a preactivation and each h is an activation after applying the rectified linear unit, or ReLU: retain positive values and replace negative values by zero.

The scalar ℓ is the logit. Sigmoid converts it to a probability p between zero and one. The predicted label is positive when $p \geq 0.5$. This threshold is held fixed for the paired decision comparisons.

Training uses CPU float32, 30 epochs, batches of 256 and Adam with learning rate 0.001 and binary cross entropy with logits. The later editing stage leaves training finished; it proposes bounded changes and evaluates forward passes. Float32 means coefficients and inference arithmetic use 32-bit floating-point values.

07 · Lesion and intervention surface

A permanent mask sets the boundary

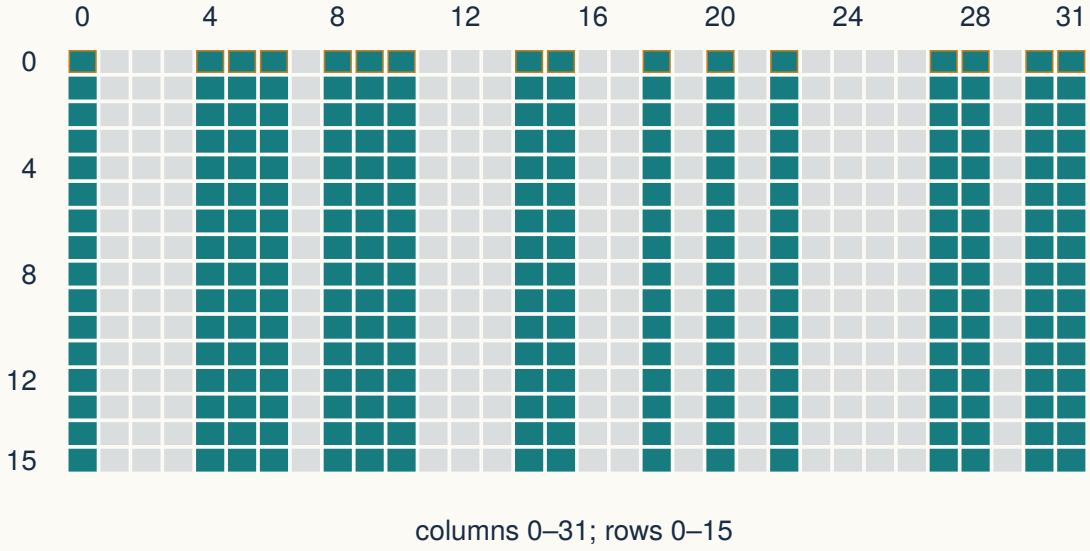


Figure 6. Actual second-layer matrix in zero-based coordinates: 16 rows by 32 columns. Gray cells are 256 permanently zero weights; teal cells are the 256 surviving eligible weights. Amber outlines identify the 16 coordinates changed by each selected main edit. This is a weight mask, not an activation map.

The calibration procedure tries zeroing 10%, 25% and 50% of second-layer columns in a fixed random order, seeking a balanced-accuracy drop of at least 0.05. It never meets that damage criterion. The retained 50% lesion records `insufficient_damage=true`; its calibration balanced accuracy is approximately 0.794938. The damaged model nevertheless defines the common origin for this editing comparison.

Only surviving `fc2.weight` coefficients can change. Masked weights stay zero; biases and other layers stay fixed. Adult permits at most 10% of all network parameters to differ and a delta norm no larger than 5% of the editable origin tensor's norm. With 3,905 total parameters, the count ceiling is 390; the surviving surface itself has 256 scalars.

For origin W^0 and candidate W', $\Delta W = W' - W^0$. The Frobenius norm $\|\Delta W\|_F = \sqrt{\sum_{r,c}(\Delta W_{rc})^2}$ combines scalar changes into one distance. Relative norm divides by $\|W^0\|_F$. Validation counts actual unequal values after float32 casting and measures the executed subtraction with double-precision norm aggregation. These are distances in parameter space, not between people.

08 · Editor interface and repeatable method

Inspect, propose, evaluate, freeze

An editor sees permitted numerical observations, the action schema and its own development feedback. It can inspect bounded information and request an evaluation within its allowance. The controller checks tensor names, indices, finite values, mask compliance, executed changed count and norm. Every evaluation reconstructs the candidate from the immutable origin.

Structured paraphrase of the prompt contract

The following explains the interface; it is not a quotation or an inferred reasoning trace. Inspect permitted target information. Return a structured JSON action using sparse additions or selected-column scaling. Request only permitted development evaluation. Use feedback to propose a later candidate. Freeze selection before final assessment. Adult diagnostics and feedback are distinct; language-model inspection includes bounded tensor slices and preset training probes, without validation example text or final rows.

Exact resource texts are public in `prompts/v1` and `prompts/v2`; controller and schema sources define the accepted actions. Sparse additions identify row, column and delta. Column scaling identifies columns and a factor. A controller can reject an attempt without producing a score: a missing score is never evidence of poor measured accuracy.

Cooperation is an information treatment

Independent Sol and Astra do not see each other's attempts. Joint receives their development histories, then runs five fresh proposal–review rounds. The reviewer may accept the full patch, revise it or reject it to no-op. Only the reviewed choice is evaluated; unmeasured drafts are not candidates with invented scores.

The final feedback cannot choose a patch, stopping point or winning editor. Selection uses the ordered rule already stated. To repeat the analysis, retain that rule as well as the exact prompt, origin, dtype, panels and schemas. Changing any of those would define a different experiment.

09 · Exact submitted changes

Sixteen coefficients, three endpoints

All selected main Adult edits change the same 16 surviving weights in row zero of the second-layer matrix. A coordinate (r, c) identifies the destination unit r and source activation c, starting at zero. The following numbers are submitted decimal deltas, before execution rounding.

Row, column (zero-based)	Sol Δ	Astra Δ	Joint Δ
(0, 0)	0.0315	0.003	0.029295
(0, 4)	0.0518	0.003	0.048174
(0, 5)	0.028	0.003	0.02604
(0, 6)	0.0252	0.003	0.023436
(0, 8)	0.0358	0.003	0.033294
(0, 9)	0.0346	0.003	0.032178
(0, 10)	0.0532	0.003	0.049476
(0, 14)	0.0417	0.003	0.038781
(0, 15)	0.028	0.003	0.02604
(0, 18)	0.0116	0.003	0.010788
(0, 20)	0.029	0.003	0.02697
(0, 22)	0.028	0.003	0.02604
(0, 27)	0.0307	0.003	0.028551
(0, 28)	0.0211	0.003	0.019623
(0, 30)	0.0488	0.003	0.045384
(0, 31)	0.0472	0.003	0.043896

Table 1. Submitted additions to the damaged-origin second-layer matrix, for selected Sol, Astra and joint patches. Rows identify all 16 changed coordinates; entries are weight units, not accuracy changes. Joint's submitted additions are exactly 0.93 times Sol's at each coordinate, while Astra adds 0.003 throughout.

Sol selects candidate 6, Astra 9 and joint 5. All 16 selected additions survive float32 casting. The common coordinate pattern does not imply identical predictions: magnitudes differ, and downstream nonlinearities and thresholds determine the effect.

The 0.93 relationship is an exact property of submitted patches. It is useful evidence about the intervention selected by cooperation, without claiming access to an editor's private reasoning or a general cooperative mechanism.

10 · From a scalar to a prediction

What does one changed connection do?

Consider coordinate $(0, 15)$: the connection from first-layer activation 15 into second-layer unit zero. Its actual saved Sol values are

$$W^{0}_{0,15} = 0.1248217299580574,$$
$$W^{\text{Sol}}_{0,15} = 0.15282173454761505.$$

For a *synthetic arithmetic illustration*, let the incoming activation $h^{(1)}_{15} = 2$. This is an illustrative value, not a reconstructed held-out activation. The connection contributes approximately 0.249643460 before the edit and 0.305643469 after it: a preactivation difference of approximately 0.056000009.

Holding first-layer activations fixed is legitimate because the edit changes only the second layer. The full preactivation difference is

$$\Delta z^{(2)}_0 = \sum_{c \in C} (W^{\text{Sol}}_{0c} - W^{0}_{0c}) h^{(1)}_c,$$

where C is the set of 16 selected columns. ReLU then gives $h'^{(2)}_0 = \max(0, z^{(2)}_0 + \Delta z^{(2)}_0)$. The output logit depends on that activation and the fixed final-layer coefficient, before sigmoid yields p'.

Executed statistic	Sol	Astra	Joint
Changed scalars	16	16	16
Delta L2	0.143815856	0.011999997	0.133748744
Relative L2	0.048917676	0.004081692	0.045493438

Table 2. Executed selected Adult patches relative to the damaged origin. Changed scalars are counts; L2 is absolute parameter-change norm and relative L2 divides by the origin second-layer norm. These describe intervention size, not performance; all three patches satisfy the same legal bounds.

The synthetic multiplication explains how a real weight participates in computation. Saved final probabilities, rather than this illustration, establish what happened to actual anonymous test decisions.

11 · Development history

Cooperation selects from its own rounds

Sol and Astra start independently from the damaged origin. Candidate indices identify saved attempts, not elapsed time or gradient steps. Their selected development balanced accuracies are 0.786946378 for Sol and 0.784309901 for Astra. Joint's selected value is 0.787596573. These scores use the fixed 2,048-row feedback panel.

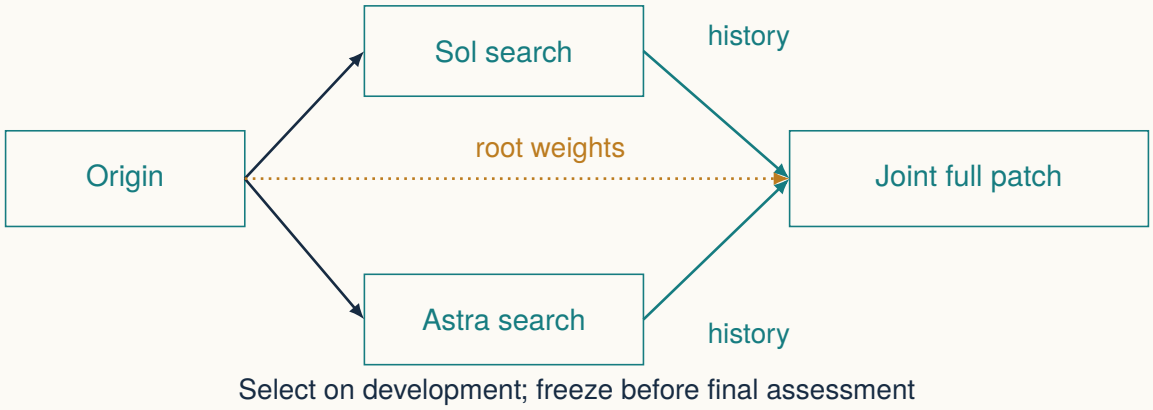


Figure 7. Adult search relationship. Independent development histories inform joint proposal/review; the origin remains unchanged as the source for every complete patch. Five fresh joint rounds are evaluated without final feedback. Arrows do not imply serial application of selected solo patches.

Joint's five scored nonzero rounds follow indices 1 through 5, plus candidate zero. Its first evaluated patch matches Sol's selected pattern; later rounds explore related magnitudes. The selected fifth patch uses 0.93 times Sol's submitted additions. The appendix retains all saved Adult scored indices and their development values, including gaps rather than inferred measurements.

Astra's development tie between candidates 8 and 9 is resolved by smaller norm. Sol's tie between candidates 6 and 7 also favors smaller norm. These examples show why primary-score ties do not leave the winner to post-hoc preference.

Joint finishes with higher development balanced accuracy than Sol, but lower final balanced accuracy. Development guides a search; it cannot guarantee that the same ordering appears on unseen cases. The relevant comparison is therefore the frozen final paired difference, rather than a favorable development trajectory alone.

12 · Metrics before the main result

Two meanings of a correct classifier

For a true positive class of size $N_+ = 3846$ and a true negative class of size $N_- = 12435$, let TP count correctly predicted positives and TN correctly predicted negatives. Positive recall is TP/N_+; negative recall is TN/N_-. Then

$$\text{BA} = \frac{1}{2}\left(\frac{TP}{N_+} + \frac{TN}{N_-}\right), \qquad \text{Accuracy} = \frac{TP + TN}{N_+ + N_-}.$$

Balanced accuracy, abbreviated BA, gives each class half the total weight. Ordinary accuracy gives each example equal weight, so the more numerous negatives carry more total weight. Both are fractions between zero and one; multiplying by 100 expresses a percentage. A difference of 0.01 is one percentage point, abbreviated pp.

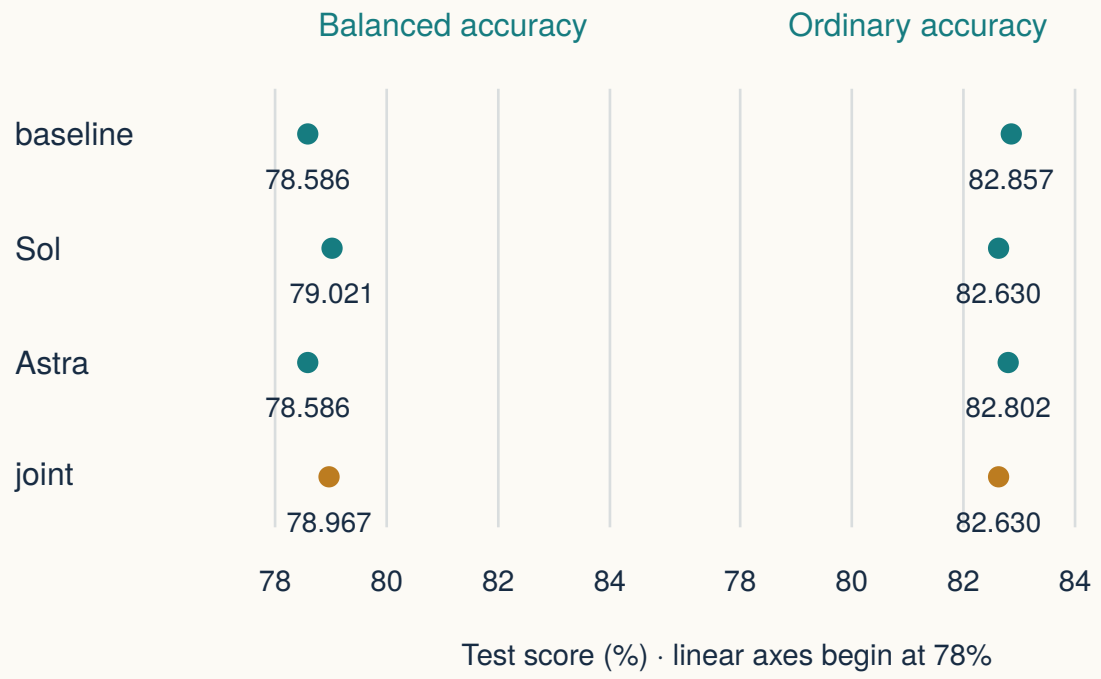


Figure 8. Selected Adult final scores on the same 16,281 original test rows, shown as percentages. Higher is better on both axes. Explicit linear axes begin at 78%, so dots encode position rather than zero-based bar area. Sol and joint improve class-balanced performance while losing ordinary accuracy.

These are two summaries of the same decisions, not competing estimates of one unnamed quality. If an edit correctly identifies more positive cases but wrongly labels additional negatives, BA can rise while ordinary accuracy falls. Which objective is appropriate depends on how a task values the errors; this experiment declares BA primary and retains the other scores.

The next table gives full displayed percentages and total correct counts. Definitions come first so the difference cannot be mistaken for inconsistent measurement.

13 · Adult main results

The improvement is metric-specific

Condition	Balanced accuracy (%)	Accuracy (%)	Correct / 16,281
baseline	78.585785	82.857318	13490
Sol	79.020686	82.630060	13453
Astra	78.585515	82.802039	13481
joint	78.966808	82.630060	13453

Table 3. Four frozen Adult endpoints on the identical ordered original test panel: n=16,281, including 3,846 positives and 12,435 negatives. BA and ordinary accuracy are percentages, with higher better; the last column counts correct decisions at $p \geq 0.5$. Sol and joint gain BA while each loses 37 correct decisions overall relative to baseline.

Sol minus baseline is 0.4349 pp in BA, with paired 95% interval 0.2138 to 0.6609 pp. Ordinary accuracy falls from approximately 82.8573% to 82.6301%. The joint endpoint has the same total correct count as Sol, but a different class composition of errors and therefore a different BA.

Astra remains close to baseline: its slight BA difference is visible in the exact numerical binding, even when a rounded plot makes the positions appear identical. Reporting all conditions prevents a narrative based only on the largest favorable difference.

What is actually compared?

Baseline is the damaged common origin, not the unlesioned clean model. Each edited endpoint is a complete intervention from that origin, selected by development before final measurement. The displayed ordering is an outcome of this seed and session, rather than a ranking of general editor ability.

The point estimates answer whether a selected patch changed performance on this saved panel. To understand why the scores moved, we next examine paired correctness transitions. To assess whether the result amounts to repair or cooperative superiority, we then bring in probability quality, controls and direct paired uncertainty.

14 · Paired decisions and exact algebra

65 rescues and 102 regressions

A rescue changes a wrong decision into a correct one; a regression changes a correct decision into a wrong one. These are paired counts because both predictions refer to the same example. Sol rescues 65 positive examples and regresses 102 negative examples, with no transitions in the opposite class-specific directions.

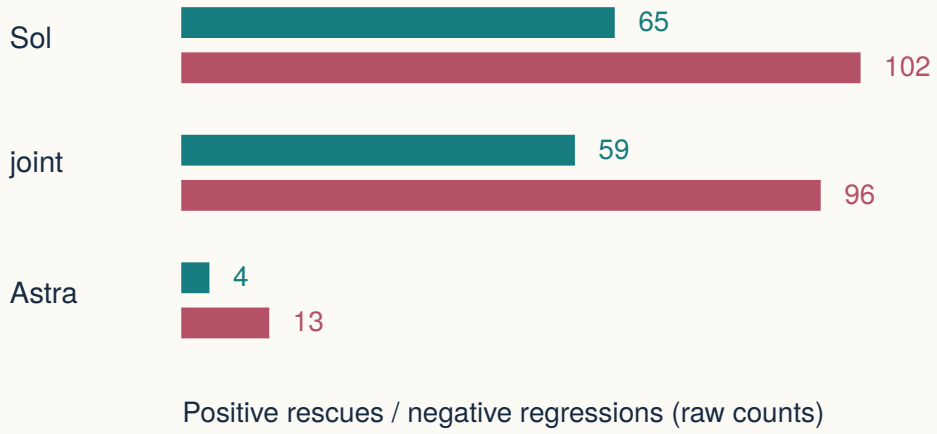


Figure 9. Paired correctness changes against damaged baseline on the 16,281-row test panel. Teal shows positive rescues; rose shows negative regressions. Lengths encode raw example counts on a common scale, not percentage points. Joint has 59 positive rescues and 96 negative regressions; Astra has 4 and 13.

$$\Delta \mathrm{BA}_{\mathrm{Sol}} = \frac{1}{2}\left(\frac{65}{3846} - \frac{102}{12435}\right),$$
$$\Delta \mathrm{Accuracy}_{\mathrm{Sol}} = \frac{65 - 102}{16281} = \frac{-37}{16281}.$$

The first expression yields 0.4349 pp; the second yields -0.2273 pp. Sixty-five rescues matter more per case to BA because positives are less numerous. Ordinary accuracy instead registers the net loss of 37 correct decisions, from 13,490 to 13,453.

Change	Anonymous ordinal	Truth	Baseline p	Sol p
Rescue	54	1	0.492739946	0.509475291
Regression	77	0	0.495158732	0.507810116

Table 4. Two saved anonymous test decisions in original order: ordinal 54 is a positive rescue and ordinal 77 a negative regression. Truth is the binary income label; p is positive-class probability. Both cross the fixed 0.5 threshold upward, helping the positive case and harming the negative case. No feature row or held-out activation is reconstructed.

15 · Controls and probability quality

A balanced gain is not restoration

ROC-AUC measures the probability ranking of positives above negatives, allowing for ties. Log-loss averages the negative log probability assigned to the true class; confident wrong predictions incur a large penalty. Brier score averages squared probability error. Higher is better for BA, accuracy and ROC-AUC; lower is better for log-loss and Brier.

State	BA	Accuracy	ROC-AUC	Log-loss	Brier
baseline	0.785858	0.828573	0.889869	0.378285	0.120487
Sol	0.790207	0.826301	0.889981	0.382725	0.122237
Astra	0.785855	0.828020	0.889885	0.378581	0.120608
joint	0.789668	0.826301	0.889975	0.382383	0.122106
clean	0.765869	0.856336	0.907734	0.314733	0.100082
random	0.786165	0.828082	0.889981	0.378680	0.120655
grid	0.785657	0.828266	0.889862	0.376637	0.120089

Table 5. All seven frozen Adult states on the same 16,281 original test examples. Entries are unitless fractions rounded to six decimals; BA/accuracy/ROC-AUC are higher-better and log-loss/Brier lower-better. Clean improves ordinary accuracy and probability quality over damaged baseline but has lower BA. Sol and joint shift class errors while worsening probability losses.

The clean state is the pre-lesion network. It outperforms damaged baseline in ordinary accuracy, AUC and both probability losses, yet has lower BA. Together with the unmet calibration damage criterion, this makes a BA recovery percentage inappropriate: clean-minus-damaged BA would be a negative denominator. We compare endpoints rather than describing a restored clean decision rule.

Random and grid are frozen development-selected controls, not favorable perturbations chosen after seeing the test. Random selects a 64-scalar patch; grid selects surviving-column scaling by 1.02. Their retained results help contextualize changes available on this small parameter surface. They do not isolate all effects of search effort or establish superiority of language-model reasoning across sessions.

The probability scores add another qualification to the Adult result: a better balance of thresholded class decisions does not imply better probability estimates. A result table with only BA would miss this tradeoff.

16 · Paired uncertainty

Does cooperation exceed Sol?

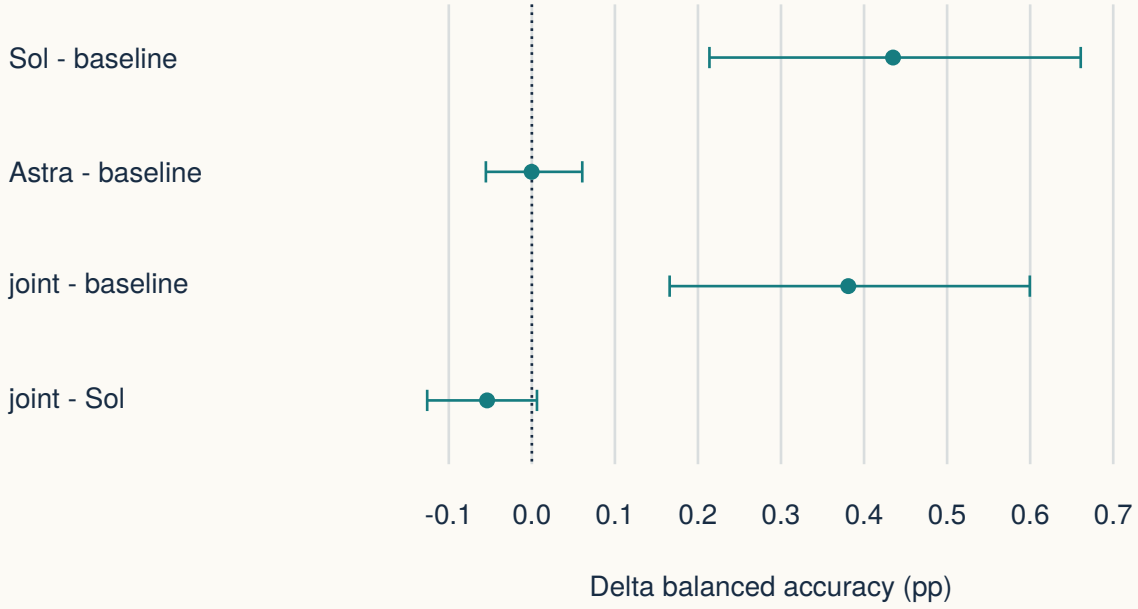


Figure 10. Adult balanced-accuracy differences in percentage points on a linear common axis. Dots mark observed paired contrasts; bars mark 95% class-stratified paired bootstrap intervals from 2,000 draws, seed101. The final row directly compares joint with Sol: its interval crosses zero, so this session does not establish cooperative superiority.

Contrast	ΔBA (pp)	95% low	95% high
Sol – baseline	+0.4349	+0.2138	+0.6609
Astra – baseline	-0.0003	-0.0554	+0.0607
joint – baseline	+0.3810	+0.1659	+0.5996
joint – Sol	-0.0539	-0.1260	+0.0062

Table 6. Exact displayed paired Adult contrasts on n=16,281 original test cases. Difference and interval endpoints use pp; positive means the first-named state has higher BA. The joint-minus-Sol point estimate is negative and its interval includes zero, unlike the positive Sol-minus-baseline interval.

Each bootstrap draw samples examples with replacement separately within each true class, retaining class sizes. The same drawn indices are used for both states in a contrast. This pairing preserves whether two methods succeed or fail on the same cases. The interval endpoints are the 2.5th and 97.5th percentiles of 2,000 difference draws.

The intervals describe paired example uncertainty under this resampling procedure. They do not estimate variability across editor sessions, training seeds, prompts or model families. Comparing marginal intervals would not replace the direct joint-minus-Sol contrast shown here.

17 · Second target and task definitions

An intact compact language model

SmolLM2-1.7B base is a compact decoder language model with approximately 1.7 billion parameters. It assigns probabilities to token continuations. Unlike the Adult target, it is intact: no artificial lesion is applied and no damaged-model recovery claim is appropriate.

ARC-Easy is the easy partition of the AI2 Reasoning Challenge, a multiple-choice grade-school science benchmark. PIQA, Physical Interaction Question Answering, tests physical commonsense with answer choices. In this protocol ARC supplies development information; PIQA is final-only. Their outcomes remain separate because they test different tasks.

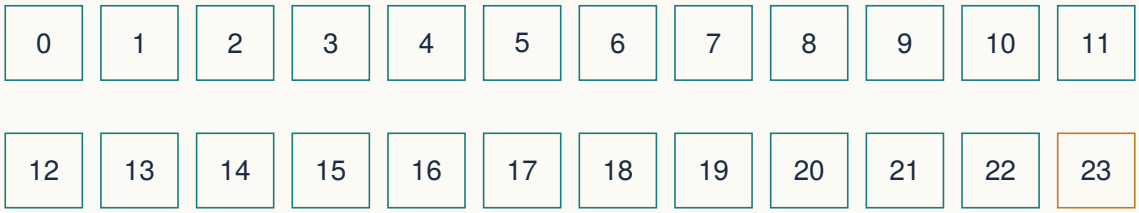


Figure 11. SmolLM2's 24 decoder blocks, indexed 0–23. The last block is highlighted to locate a real nonselected intervention. This is an architecture schematic, not a display of all 1.7 billion coefficients or a measured activation trace.

ARC train128 supplies diagnostics; validation128 supplies development feedback. Frozen final panels contain ARC512 and PIQA256 examples. Native task definitions determine the question context, continuation, choice ordering and gold label. No benchmark question text is reproduced here.

The runtime is MPS float32 with eager attention, batch size one, float32 softmax and no logits cache. Main results use corrected-prompt v2. The earlier prompt described float16 despite actual float32 execution; v2 corrects that description. v2 has fresh editor sessions but reuses panels already exposed in v1, after the analyst saw earlier results. It is not an independent replication. A train8 engineering check does not establish whole-benchmark numerical equivalence; no float32 train64 gate was run.

18 · Transformer mathematics

Inside one gated MLP

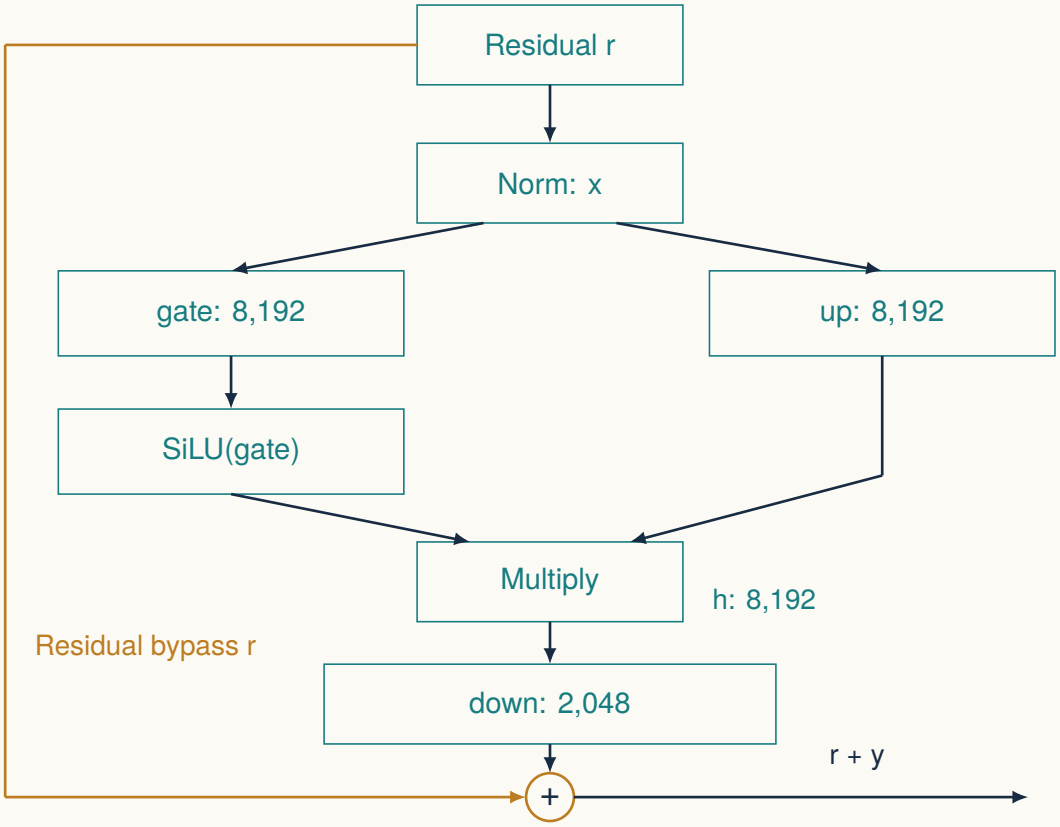


Figure 12. One gated multilayer perceptron inside a transformer block: residual width 2,048 expands to 8,192 features and returns to 2,048. Separate gate and up projections meet through coordinatewise multiplication; a residual bypass adds the input back. Geometry is schematic; no fabricated coefficient or activation values are shown.

For incoming residual $r \in \mathbb{R}^{2048}$, let $x = \mathrm{RMSNorm}(r)$. RMSNorm means root mean square normalization, with learned rescaling. Define

$$g = W_g x, \quad u = W_u x, \quad h = \mathrm{SiLU}(g) \odot u, \qquad r' = r + W_d h.$$

W_g and W_u map 2,048 coordinates to 8,192; g is the gate vector and u the up-projected vector. SiLU is the sigmoid linear unit, $\mathrm{SiLU}(t) = t/(1+e^{-t})$, applied coordinatewise. The symbol $\odot$ multiplies corresponding coordinates. W_d maps the gated features h back to 2,048 coordinates, and r' is the residual after adding this MLP contribution.

Only down-projection weights W_d are eligible for editing. A column controls how one expanded feature contributes across output coordinates. Changing that column changes the shared map, while its actual effect still depends on the input's gated features. The real intervention on the next page uses columns, not invented individual LM scalar samples.

19 · Real language-model interventions

A legal proposal can remain unselected

LM proposals may scale selected down-projection columns or add sparse scalar deltas. Actual changed scalars cannot exceed 0.1% of unique model parameters; each changed tensor must remain within 5% relative Frobenius change. Counts and norms describe executed, post-cast changes from the common intact origin.

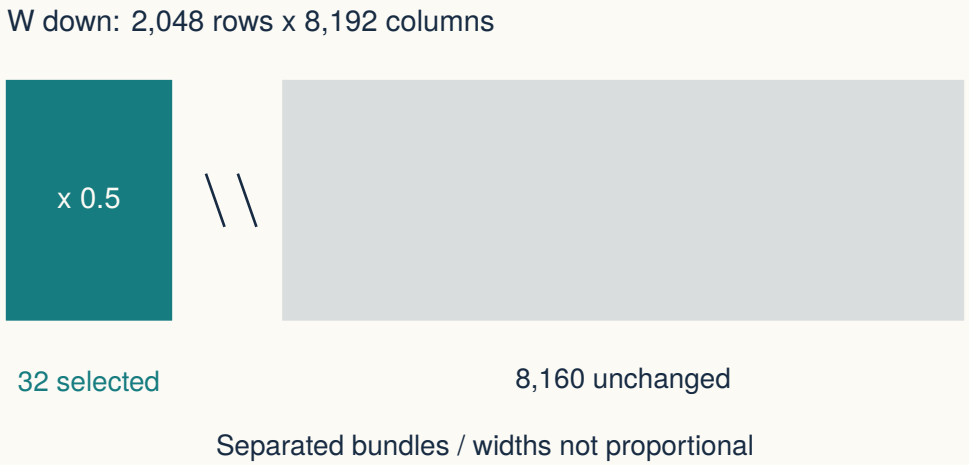


Figure 13. Actual Astra candidate 1 targets the last block's down projection: all 2,048 rows in columns 0–31 are multiplied by 0.5. Bundles are separated and widths are explicitly not proportional. The operation changes 65,536 weights; the candidate is nonselected, so this image does not describe the final endpoint.

The exact tensor is `model.layers.23.mlp.down_proj.weight`; block 23 is the last of 24 zero-indexed blocks. Thirty-two columns times 2,048 rows gives 65,536 actual changes. Its measured delta norm is 13.910684498150486 and relative norm is 0.030596354. This is a genuine nonzero intervention satisfying the allowed surface and size constraints.

Raw before-and-after scalar samples for this LM candidate were not retained in approved evidence. The patch operation, measured count, norms and executed tensor hashes establish its geometry without inventing plausible-looking coefficients.

An intervention can be legal and nonzero yet fail to improve the primary development score. It can also tie the unchanged model. Since the selection rule prefers smaller norm at equal score, a nonzero tie loses to candidate zero. Selection and proposal generation are distinct empirical events.

20 · Answer likelihood and normalization

What makes one answer the prediction?

Let q denote question context and $a_{j,t}$ token t of answer continuation j. If continuation j has T_j tokens, its likelihood score is

$$L_j = \sum_{t=1}^{T_j} \log P_\theta(a_{j,t} \mid q, a_{j,<t}).$$

P_θ is the target language model with parameters θ; $a_{j,<t}$ means earlier tokens in that answer. The score adds token log probabilities. Larger, or less negative, means the whole continuation has greater conditional likelihood under this scoring convention.

$$\hat{j}_{\text{raw}} = \arg\max_j L_j, \qquad \hat{j}_{\text{norm}} = \arg\max_j \frac{L_j}{C_j}.$$

Argmax means choose the index with the greatest score. C_j is the Python character length of the choice text: **not token count**. A tie takes the first maximum in native choice order. Raw accuracy, `acc`, and character-normalized accuracy, `acc_norm`, each average whether its chosen index equals the gold label.

Synthetic scoring illustration

Suppose invented choices A and B have log-likelihood sums -4 and -6, and character lengths 4 and 12. Raw choice is A because $-4 > -6$. Normalized scores are -1 and -0.5, so normalized choice is B. These numbers explain the scoring rule; they are not dataset results or copied question text.

The native harness constructs contexts and continuations using the pinned task definitions. The public numeric records retain likelihoods, character/token lengths and gold choices so both conventions can be recomputed offline. Token lengths remain audit information, without becoming the normalization divisor.

21 · Language-model development

Candidate zero wins the tie-break

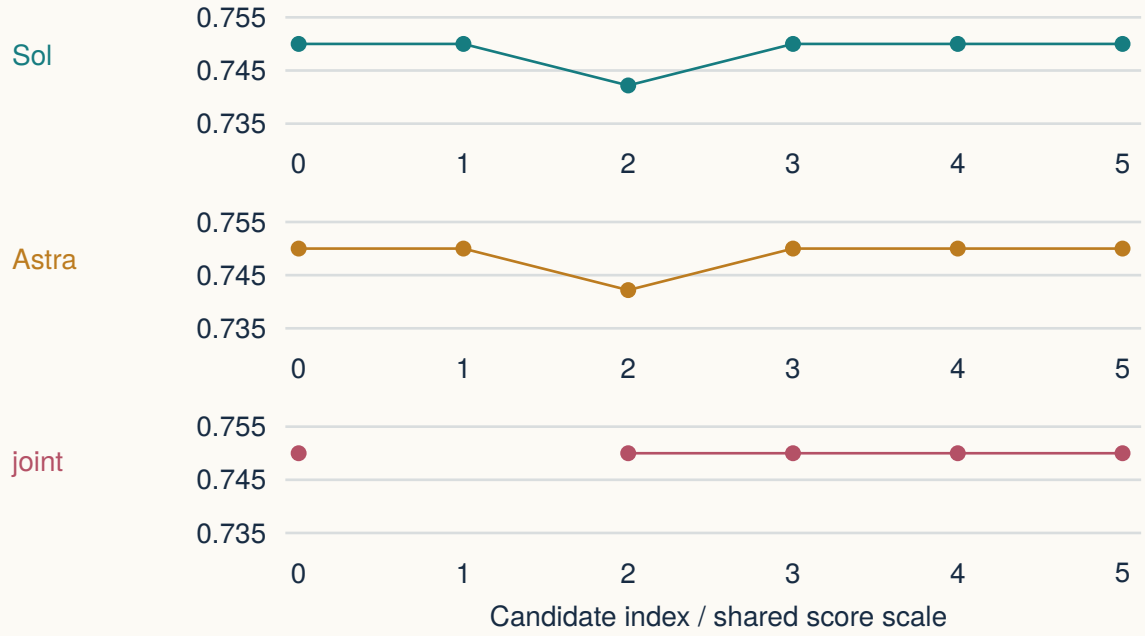


Figure 14. Main corrected-prompt v2 ARC development character-normalized accuracy, on the fixed 128-row feedback panel. Each row uses the same fraction scale; dots show saved scored indices. Joint attempt 1 is invalid and unscored, so the visible gap is never interpolated. Higher is better, but equal scores prefer a smaller executed change.

Candidate	Sol acc_norm	Astra acc_norm	Joint acc_norm
0	0.7500000	0.7500000	0.7500000
1	0.7500000	0.7500000	invalid / unscored
2	0.7421875	0.7421875	0.7500000
3	0.7500000	0.7500000	0.7500000
4	0.7500000	0.7500000	0.7500000
5	0.7500000	0.7500000	0.7500000

Table 7. Saved development scores for candidate zero and five opportunities per condition. Scores are acc_norm fractions on ARC development128; higher is better. Joint index 1 has no measured score. All valid nonzero candidates tie or underperform zero's 0.75, so the zero-norm unchanged candidate wins selection.

Sol and Astra each measure five nonempty candidates. Joint makes five attempts, of which indices 2–5 are scored. The recorded development trajectory therefore demonstrates actual search, even though every selected patch is empty.

22 · Completed language-model results

Four names, one exact weight state

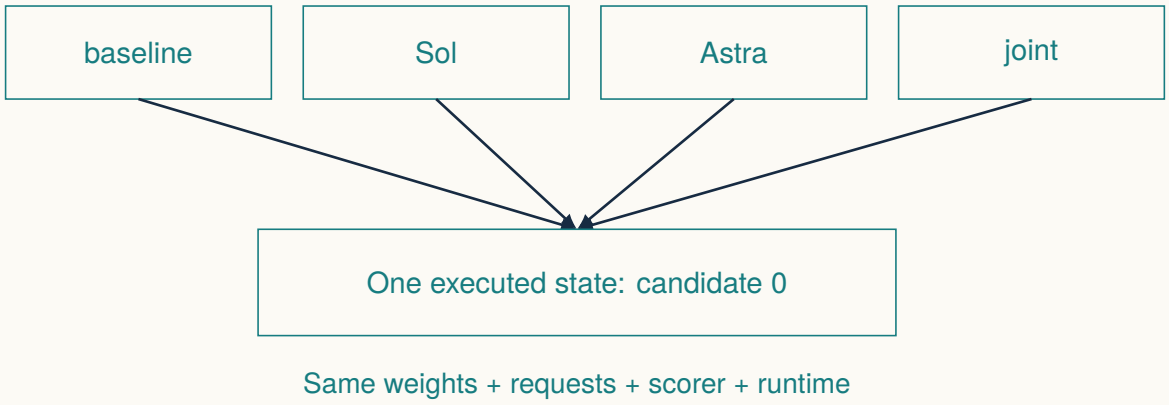


Figure 15. Baseline, Sol, Astra and joint all select candidate zero in main v2. Matching actual weights, origin, ordered panels, requests, scorer and runtime identifies one executed group with shared final predictions. Four condition names are therefore not four independent final observations.

Task	n	Character-normalized	Raw likelihood
arc_easy	512	376/512 = 73.437500%	396/512 = 77.343750%
piqa	256	201/256 = 78.515625%	205/256 = 80.078125%

Table 8. Completed main corrected-prompt v2 final panels: ARC-Easy n=512 and PIQA n=256. Columns show exact correct fractions and percentage accuracy under character-normalized and raw continuation likelihood; higher is better. Every listed task result applies identically to baseline, Sol, Astra and joint because their selected weights and predictions are shared.

ARC-Easy yields 376/512 correct, or exactly 73.4375%, under character normalization; raw likelihood yields 396/512, exactly 77.34375%. PIQA yields 201/256, exactly 78.515625%, normalized and 205/256, exactly 80.078125%, raw. Different scoring conventions choose answers differently from the same saved likelihoods.

All paired selected-method differences are exactly zero: there are no selected rescues or regressions against baseline. This follows from exact identity, rather than four independent experiments converging to numerically similar scores.

Main v2 PIQA is complete. Historical v1's original PIQA failure before scoring and later supplement belong to version history; they do not replace or invalidate this completed v2 table. The selected null intervention is the endpoint under these bounds and development rule, not a proof that every possible direct edit is ineffective.

23 · Discussion and reproduction

What the evidence adds, and how to inspect it

The first question receives different answers on the two targets. Adult's bounded edit improves the declared primary metric while reducing total correct decisions and worsening probability losses. The paired algebra explains the change as an error tradeoff. On intact SmolLM2, legal proposals are explored but none displaces the unchanged origin under development selection. A useful editing study must retain both kinds of outcome.

The second question concerns cooperation. Joint has no demonstrated advantage over Sol here: its Adult point estimate is lower with a direct interval crossing zero, while the LM endpoints are identical. A common origin and explicit information flow make the comparison interpretable, but one seed/session and unequal Adult opportunities cannot establish a general cooperation effect.

Recompute saved results without a model or API

From a prepared checkout with the project's numeric dependencies installed:

```
python3 -m ws_lab.reproduce verify --data-dir data
python3 -m ws_lab.reproduce analyse --data-dir data \
  --output analysis.json
python3 scripts/build_research_report.py
```

Use a new output file for analysis. `data/manifest.json` binds numeric predictions, graph weights, summaries and exact prompt resources. `configs/defaults.json` declares identities and bounds; `docs/SCORER.md` specifies native scoring. `data/lm_v2_frozen_patches.json` retains selected LM interventions. The report's HTML embeds a compact measurement receipt; PDF and HTML share this prose source.

New model replay and new editor experiments are separate explicit operations described in `docs/REPRODUCE.md`; they create fresh output and consumption identities and never reuse historical consumed assessments. Building this report performs neither operation. Public source: https://github.com/detasar/llm-weight-editing-study.

24 · Technical appendix

Saved Adult development trajectory

Editor	Index	Dev BA	Delta L2	Selection
Sol	0	0.782999312	0.000000	
Sol	2	0.783334608	0.080303	
Sol	3	0.783339708	0.144546	
Sol	4	0.784655397	0.140000	
Sol	5	0.782999312	0.140000	
Sol	6	0.786946378	0.143816	selected
Sol	7	0.786946378	0.146692	
Astra	0	0.782999312	0.000000	
Astra	2	0.782039318	0.120000	
Astra	3	0.779057345	0.120000	
Astra	4	0.783339708	0.060000	
Astra	5	0.782364415	0.100000	
Astra	6	0.783014610	0.067082	
Astra	7	0.783659706	0.040000	
Astra	8	0.784309901	0.020000	
Astra	9	0.784309901	0.012000	selected
joint	0	0.782999312	0.000000	
joint	1	0.786946378	0.143816	
joint	2	0.786946378	0.140940	
joint	3	0.787271476	0.135187	
joint	4	0.785635789	0.129434	
joint	5	0.787596573	0.133749	selected

Table 9. All saved scored Adult main-editor indices, including candidate zero, on the fixed 2,048-row development-feedback panel. Dev BA is a higher-better fraction; delta L2 measures change from the damaged origin. Gaps are not fabricated evaluations. Selection uses BA, then smaller norm, fewer changed scalars and earlier index; original allowances are ten per independent editor and five joint rounds.

25 · References and version record

Sources and measurement identity

Takehi, R.; Teoh, J.; Zhang, A.; Khattab, O. (2 November 2026). *Sparks of Weight-Space Programming: LLMs can natively read & write neural network weights*. Research blog: https://rikiyat.github.io/blog/weight-space-programming/.

Becker, B.; Kohavi, R. *Adult*. UCI Machine Learning Repository: https://archive.ics.uci.edu/dataset/2/adult.

Hugging Face / HuggingFaceTB. *SmolLM2-1.7B* base model card: https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B. Underlying SmolLM2 paper: https://arxiv.org/abs/2502.02737.

Clark et al. (2018). *Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge*. https://arxiv.org/abs/1803.05457.

Bisk et al. (2020). *PIQA: Reasoning about Physical Commonsense in Natural Language*. https://arxiv.org/abs/1911.11641.

Publication: 4 November 2026. Main LM v2 corrects float16 prompt wording to float32 and measures both final panels; earlier-exposed panels remain disclosed in Methods. Code: MIT. Report, original conceptual artwork and derived numeric data: CC BY 4.0. The reference-title excerpt retains third-party rights. Full frozen manifest binding: 8f8869a113d93000bb8fffd547258b36ab955a6bf7a7f8ddbe589fbdbd231d38.